

Validating Predictions

Last modified: 2026-10-02 01:47:22 (PDT)

Overfitting

Exercise 0.1 (Training error versus error on new cars). The built-in `mtcars` data frame has $n = 32$ cars. Fit four models for fuel efficiency `mpg` as a polynomial in weight `wt`, of degree 1, 2, 3, and 4, using only the first 20 rows as the training set. For each model, compute the root mean squared error (RMSE) on the 20 training rows and on the remaining 12 rows.

1. Which degree has the smallest training RMSE?
2. Which degree has the smallest RMSE on the 12 held-out rows?
3. Do the terms added beyond degree 2 improve predictions for the held-out cars?

Solution

Solution 0.1.

```

train_cars <- mtcars[1:20, ]
held_out_cars <- mtcars[21:32, ]

rmse <- function(model, data) {
  sqrt(mean((data$mpg - predict(model, newdata = data))^2))
}

overfitting_results <- tibble::tibble(degree = 1:4) |>
  dplyr::mutate(
    fit = purrr::map(
      degree,
      \(d) lm(mpg ~ poly(wt, d, raw = TRUE), data = train_cars)
    ),
    training_RMSE = purrr::map_dbl(fit, rmse, data = train_cars),
    held_out_RMSE = purrr::map_dbl(fit, rmse, data = held_out_cars)
  ) |>
  dplyr::select(-fit)

overfitting_results
#> # A tibble: 4 x 3
#>   degree training_RMSE held_out_RMSE
#>   <int>         <dbl>         <dbl>
#> 1     1           2.91           3.45
#> 2     2           2.46           3.18
#> 3     3           2.45           3.21
#> 4     4           2.44           3.19

best_train <- overfitting_results$degree[
  which.min(overfitting_results$training_RMSE)
]
best_held_out <- overfitting_results$degree[
  which.min(overfitting_results$held_out_RMSE)
]

```

1. The smallest training RMSE is at degree 4. Each model contains the previous one as a special case, so its training RMSE cannot increase as the degree grows.
2. The smallest held-out RMSE is at degree 2.
3. No. Going from degree 2 to degree 3 or 4 lowers the training RMSE a little but leaves the held-out RMSE essentially unchanged (it does not fall below its degree-2 value). The terms beyond degree 2 improve the fit to the 20 training cars without

improving predictions for the 12 held-out cars. Degree 1 to degree 2 is different: there the held-out RMSE also falls.

Definition 0.1 (Overfitting). **Overfitting** occurs when a model fits the training data well but predicts poorly for new observations. It results from including too many predictors relative to the effective sample size.

In Exercise 0.1, the terms beyond degree 2 fit the 20 training cars more closely but do not predict the 12 held-out cars better. The effect is small here: it is the beginning of overfitting, not a dramatic case.

Training, validation, and test sets

Exercise 0.2 (Sizes of a three-way split). The `mtcars` data frame has $n = 32$ cars. Suppose we assign $\lfloor 0.6n \rfloor$ cars to a training set, $\lfloor 0.2n \rfloor$ cars to a validation set, and all remaining cars to a test set. How many cars are in each set?

Solution

Solution 0.2. $\lfloor 0.6 \cdot 32 \rfloor = \lfloor 19.2 \rfloor = 19$ training cars, $\lfloor 0.2 \cdot 32 \rfloor = \lfloor 6.4 \rfloor = 6$ validation cars, and $32 - 19 - 6 = 7$ test cars.

Definition 0.2 (Training/validation/test split). A **training/validation/test split** divides the data into three sets:

- Use the *training set* to estimate model parameters.
- Use the *validation set* to compare candidate models, choose transformations, or tune hyperparameters.
- Use the *test set* once at the end to estimate final out-of-sample performance.

This validation approach extends the basic train/test split by separating model tuning from final model assessment. It follows James et al. (2021, 198–201).

Keeping the test set untouched during model building helps avoid optimistic bias from repeatedly trying many models. In Exercise 0.2, a rule of 60% / 20% / 20% on $n = 32$ cars gives sets of 19, 6, and 7 cars, and the 7 test cars are used only once, after the model is chosen.

Example 0.1 (Numerical example). This example (Example 0.1) uses R’s built-in `mtcars` dataset ($n = 32$ cars) to predict fuel efficiency (`mpg`) from vehicle weight (`wt`). It uses one

random split to compare linear, quadratic, cubic, and quartic models ($\text{mpg} \sim \text{wt}$, $\text{mpg} \sim \text{wt} + \text{I}(\text{wt}^2)$, $\text{mpg} \sim \text{wt} + \text{I}(\text{wt}^2) + \text{I}(\text{wt}^3)$, and $\text{mpg} \sim \text{wt} + \text{I}(\text{wt}^2) + \text{I}(\text{wt}^3) + \text{I}(\text{wt}^4)$) on a validation set, then reports the chosen model's test RMSE on untouched test data (James et al. 2021, 213).

```
set.seed(108)
n <- nrow(mtcars)

n_train <- floor(0.6 * n)
n_valid <- floor(0.2 * n)

idx_train <- sample.int(n, size = n_train)
idx_remaining <- setdiff(seq_len(n), idx_train)
idx_valid <- sample(idx_remaining, size = n_valid)
idx_test <- setdiff(idx_remaining, idx_valid)

split_sizes <- tibble::tibble(
  split = c("training", "validation", "test"),
  n = c(length(idx_train), length(idx_valid), length(idx_test))
)
```

Table 1: Sizes of the training, validation, and test splits

```
split_sizes
#> # A tibble: 3 x 2
#>   split      n
#>   <chr>   <int>
#> 1 training    19
#> 2 validation     6
#> 3 test        7
```

```

train_dat <- mtcars[idx_train, ]
valid_dat <- mtcars[idx_valid, ]
test_dat <- mtcars[idx_test, ]

model_linear <- lm(mpg ~ wt, data = train_dat)
model_quadratic <- lm(mpg ~ wt + I(wt^2), data = train_dat)
model_cubic <- lm(mpg ~ wt + I(wt^2) + I(wt^3), data = train_dat)
model_quartic <- lm(
  mpg ~ wt + I(wt^2) + I(wt^3) + I(wt^4),
  data = train_dat
)

compute_rmse <- function(model, data) {
  sqrt(mean((data$mpg - predict(model, newdata = data))^2))
}

candidate_models <- list(
  linear = model_linear,
  quadratic = model_quadratic,
  cubic = model_cubic,
  quartic = model_quartic
)

validation_results <- tibble::tibble(
  model = names(candidate_models)
) |>
dplyr::mutate(
  training_RMSE = purrr::map_dbl(
    model,
    \(model_name) compute_rmse(candidate_models[[model_name]], train_dat)
  ),
  validation_RMSE = purrr::map_dbl(
    model,
    \(model_name) compute_rmse(candidate_models[[model_name]], valid_dat)
  )
)

model_labels <- c(
  linear = "linear model",
  quadratic = "quadratic model",
  cubic = "cubic model",
  quartic = "quartic model"
)

chosen_model_name <-
  validation_results |>
  dplyr::arrange(validation_RMSE) |>
  dplyr::slice(1) |>
  dplyr::pull(model)

chosen_model_label <- model_labels[[chosen_model_name]]

```

```

partition_colors <- c(
  training = "#1b9e77",
  validation = "#d95f02",
  test = "#7570b3"
)
partition_shapes <- c(
  training = 16,
  validation = 17,
  test = 15
)
wt_range <- range(c(train_dat$wt, valid_dat$wt, test_dat$wt))
mpg_range <- range(c(train_dat$mpg, valid_dat$mpg, test_dat$mpg))
wt_grid <- seq(wt_range[1], wt_range[2], length.out = 200)

plot_partitions_with_fit <- function(model, fit_label) {
  plot(
    train_dat$wt,
    train_dat$mpg,
    xlab = "wt",
    ylab = "mpg",
    pch = partition_shapes[["training"]],
    xlim = wt_range,
    ylim = mpg_range,
    col = partition_colors[["training"]]
  )
  points(
    valid_dat$wt,
    valid_dat$mpg,
    pch = partition_shapes[["validation"]],
    col = partition_colors[["validation"]]
  )
  points(
    test_dat$wt,
    test_dat$mpg,
    pch = partition_shapes[["test"]],
    col = partition_colors[["test"]]
  )
  fitted_curve <- predict(model, newdata = data.frame(wt = wt_grid))
  lines(wt_grid, fitted_curve, col = "blue", lwd = 2)
  legend(
    "topright",
    legend = c("training", "validation", "test", fit_label),
    col = c(unnamed(partition_colors), "blue"),
    pch = c(unnamed(partition_shapes), NA),
    lty = c(NA, NA, NA, 1),
    lwd = c(NA, NA, NA, 2),
    bty = "n"
  )
}

```

```
plot_partitions_with_fit(model_linear, "linear fit")
```

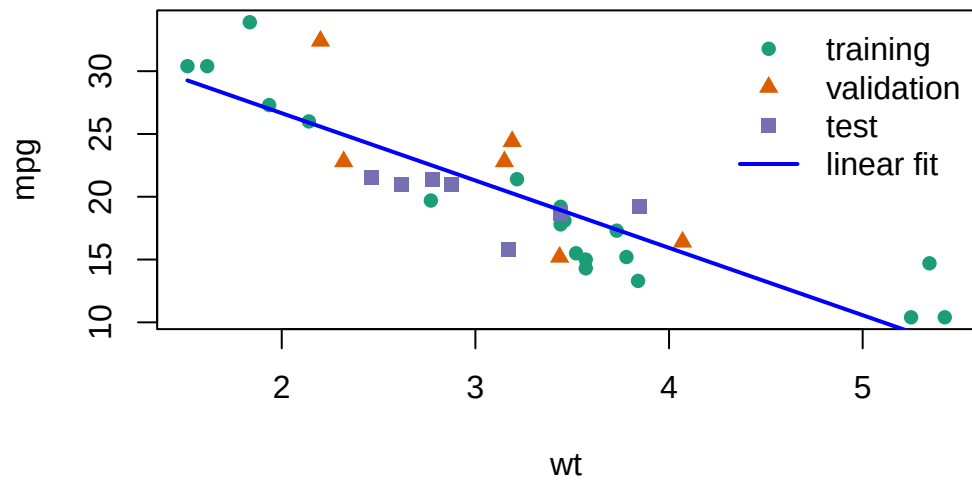


Figure 1: Linear model fit superimposed on data partitions

```
plot_partitions_with_fit(model_quadratic, "quadratic fit")
```

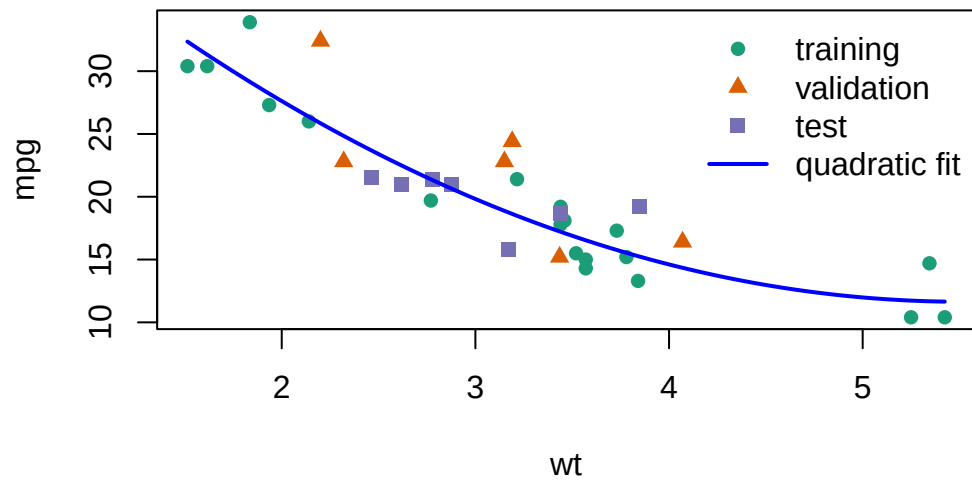


Figure 2: Quadratic model fit superimposed on data partitions

```
plot_partitions_with_fit(model_cubic, "cubic fit")
```

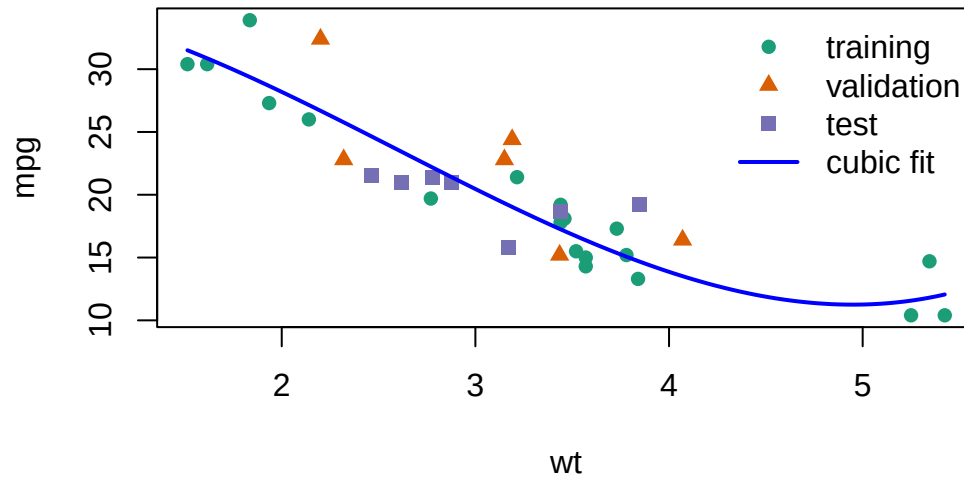


Figure 3: Cubic model fit superimposed on data partitions

```
plot_partitions_with_fit(model_quartic, "quartic fit")
```

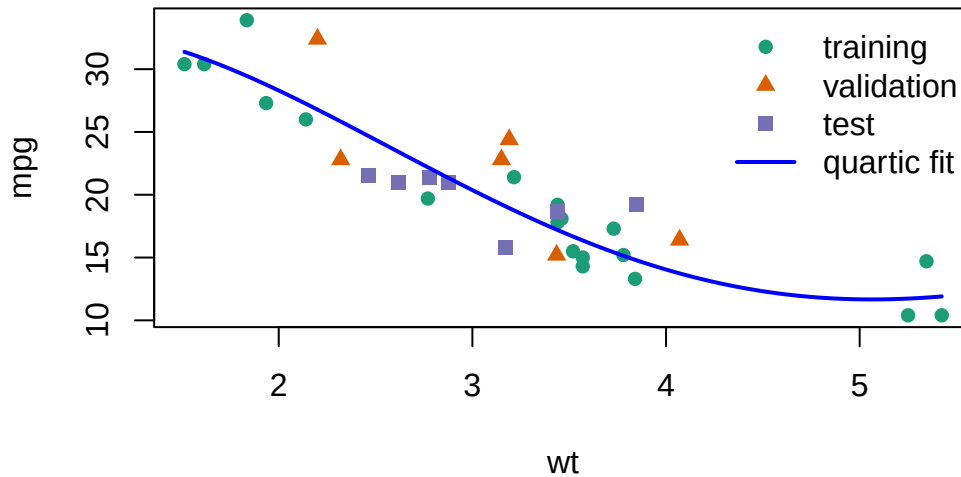


Figure 4: Quartic model fit superimposed on data partitions

Table 2: Training and validation RMSE for candidate models

```
validation_results
#> # A tibble: 4 x 3
#>   model      training_RMSE validation_RMSE
#>   <chr>          <dbl>          <dbl>
#> 1 linear          2.80            3.84
#> 2 quadratic        1.96            4.16
#> 3 cubic           1.91            3.98
#> 4 quartic          1.90            3.97
```

The chosen model is the linear model, because it has the lowest validation RMSE of the four. This split uses different cars from Exercise 0.1, and its validation set has only 6 cars, so the model it picks can differ from the one chosen there.

Table 3: Validation and test RMSE for the chosen model

```
performance_comparison
#> # A tibble: 2 x 3
#>   split      model  RMSE
#>   <chr>     <chr> <dbl>
#> 1 validation linear  3.84
#> 2 test      linear  2.44
```

k -fold cross-validation

Exercise 0.3 (Folds of the `mtcars` data). Suppose we use $k = 4$ -fold cross-validation on the $n = 32$ cars in `mtcars`.

1. How many cars are in each fold?
2. Each time a model is fit, how many cars are used to fit it?
3. How many models are fit in total?
4. How many times is a given car used for fitting, and how many times for prediction?

Solution

Solution 0.3.

1. $32/4 = 8$ cars per fold.
2. Each fit leaves out one fold, so it uses $32 - 8 = 24$ cars.
3. One model per fold, so 4 models.
4. A given car is in exactly one fold. It is used for prediction once, when its fold is left out, and for fitting in the other 3 fits.

Definition 0.3 (k -fold cross-validation). In k -fold cross-validation:

1. The data are randomly divided into k equal-sized subsets (“folds”).
2. For each fold $i = 1, \dots, k$:
 - Fit the model using all data *except* fold i .
 - Compute predicted values for the observations in fold i .
3. Compute a summary prediction-error measure across all folds.

Values of $k = 5$ or $k = 10$ are typical.

If data are limited, k -fold cross-validation can replace the single validation set of Definition 0.2 for more stable tuning. In Exercise 0.3, $k = 4$ folds of 8 cars each give 4 fits on 24 cars each.

References

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.